

ICT 1306

OBJECT ORIENTED PROGRAMMING

9.DESIGN PATTERNS

TC Irugalbandara
KHA Hettige

What is a pattern?

- Patterns = problem/solution pairs in context
- Patterns facilitate reuse of successful software architectures and design
- Not code reuse
 - Instead, solution/strategy reuse
 - Sometimes, interface reuse

Design Patterns

Someone has already solved your problems.

- Design patterns describe recurring software design problems and their solutions.
- Represent the best practices used by experienced object-oriented software developers.
- These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.
- A design pattern isn't a finished design that can be transformed directly into code.
- It is a description or template for how to solve a problem that can be used in many different situations

What is Gang of Four (GOF)?

- In 1994, four authors Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides published a book titled **Design Patterns - Elements of Reusable Object-Oriented Software** which initiated the concept of Design Pattern in Software development.
- These authors are collectively known as Gang of Four (GOF).

Usage of Design Pattern

Speed up the development process

- by providing tested, proven development paradigms.

Common platform for developers

- Design patterns provide a standard terminology and are specific to particular scenario.
- For example, a singleton design pattern signifies use of single object so all developers familiar with single design pattern will make use of single object and they can tell each other that program is following a singleton pattern.

Best Practices

- Design patterns have been evolved over a long period of time and they provide best solutions to certain problems faced during software development.
- Learning these patterns helps inexperienced developers to learn software design in an easy and faster way.

Types of Design Patterns

- **Creational** : These design patterns provide a way to create objects while hiding the creation logic. All about class instantiation.
- concern the process of object creation
 - Ex: Abstract Factory , Singleton
- **Structural** : These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.
- deal with the composition of classes or objects
 - Ex: Adapter, Bridge
- **Behavioral** : These design patterns are specifically concerned with communication between objects.
 - Ex: Observer, State

Observer Pattern

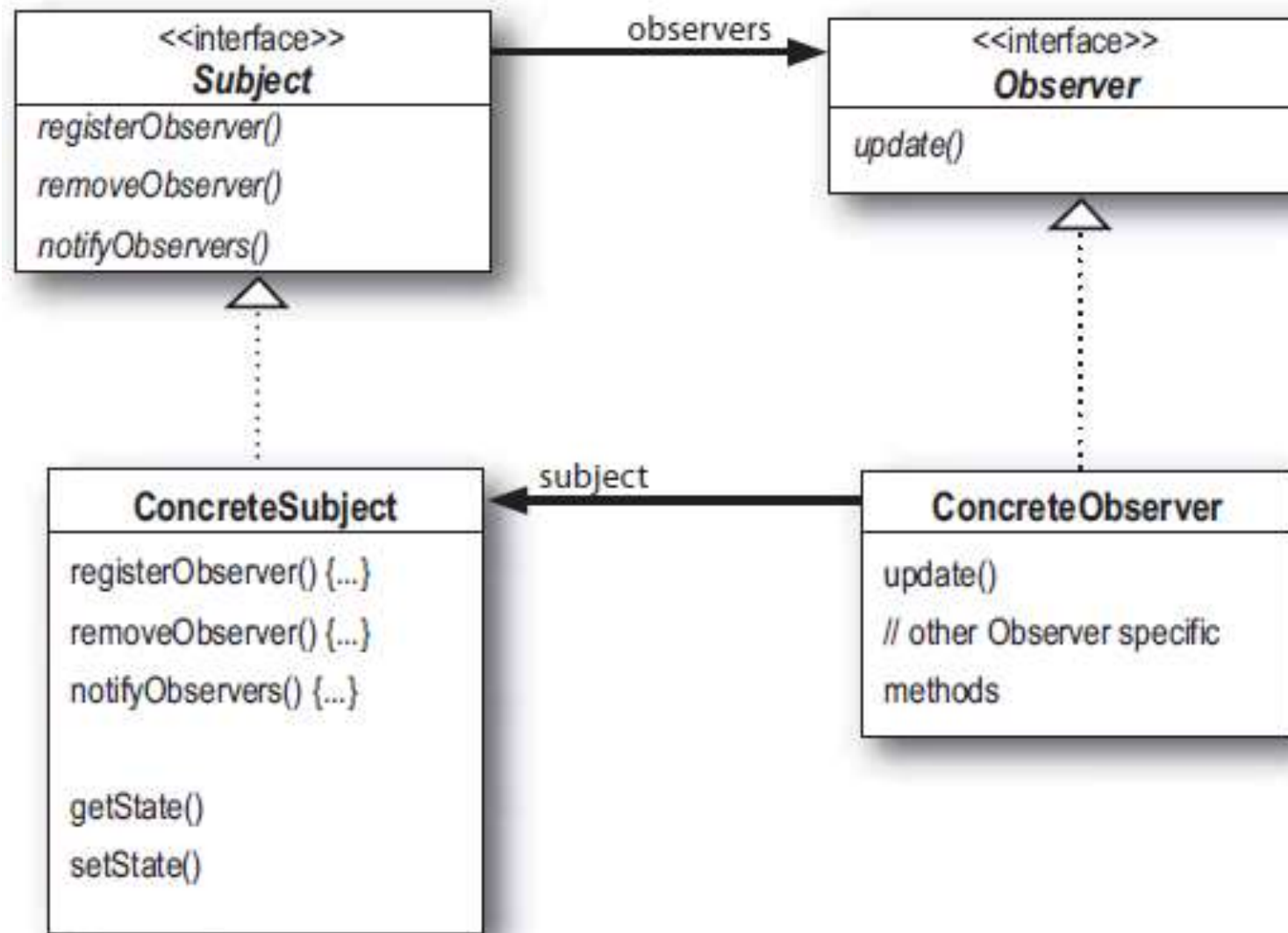
- **Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.**
- Problem
 - dependent's state must be consistent with master's state
- Solution structure
 - define four kinds of objects:
 - abstract subject
 - maintain list of dependents; notifies them when master changes
 - abstract observer
 - define protocol for updating dependents
 - concrete subject
 - manage data for dependents; notifies them when master changes
 - concrete observers
 - get new subject state upon receiving update message

Examples

- News/Event subscription

Publisher + Subscriber = Observer pattern
↑ ↑
Subject Observer

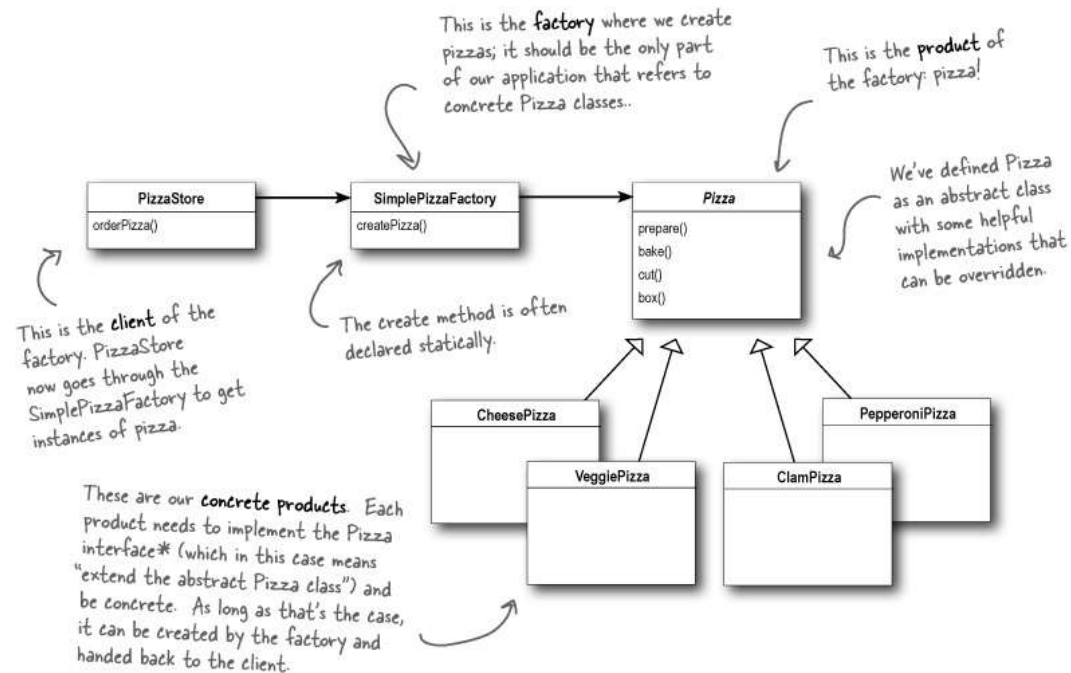
- Weather information → display current state, statistics and predict



1. We can add an observer anytime.
2. We can remove observer at anytime.
3. If new class create and need to use Subject's data , simple implement observer interface in it and need to register as a observer in the subject class.

Factory Pattern

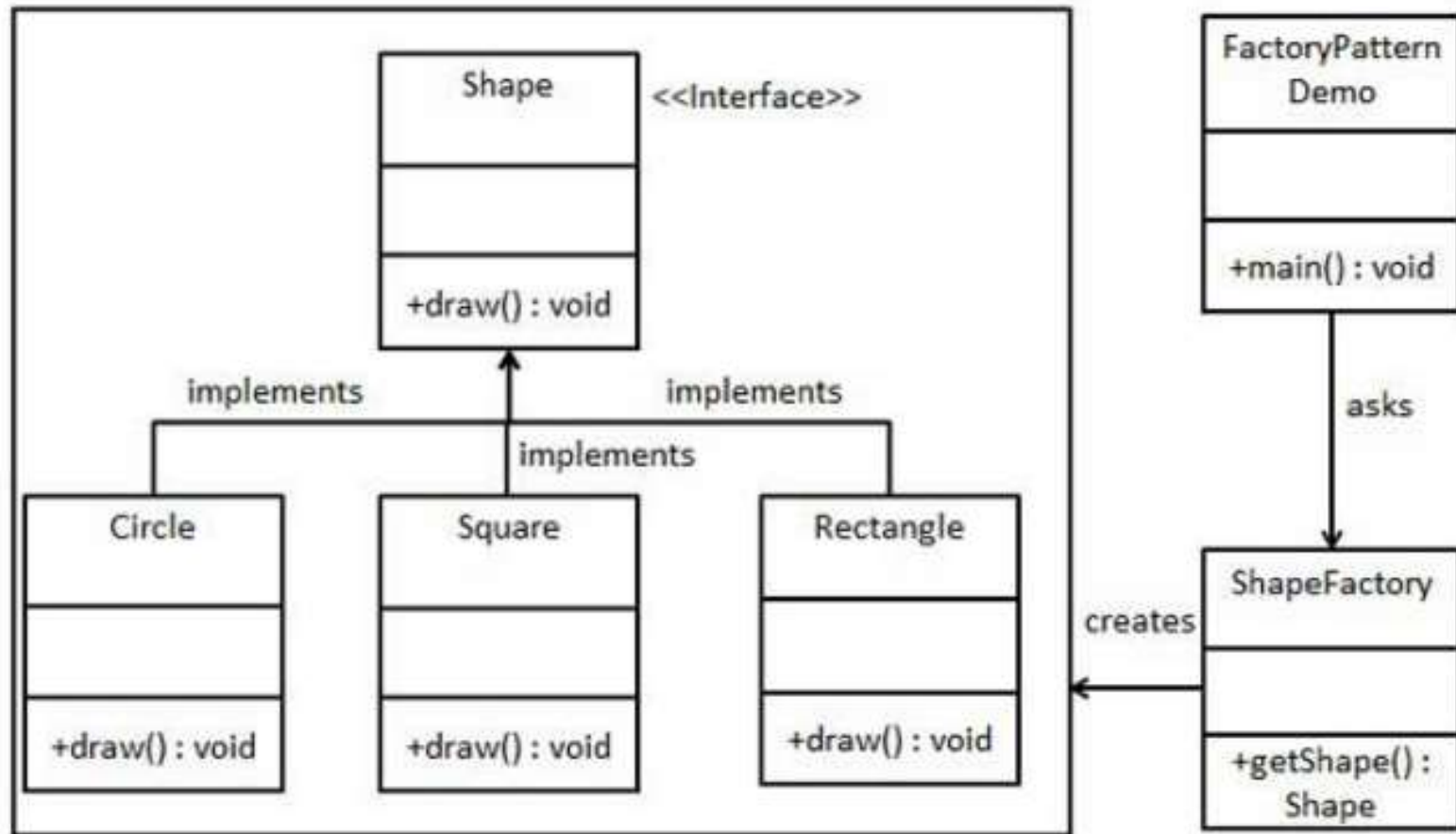
- In Factory pattern, we create object without exposing the creation logic to the client and refer to newly created object using a common interface.
- Example:
 - Pizza Hut



Factory Pattern

Example Implementation

- Create a Shape interface and concrete classes implementing the Shape interface.
- We have multiple shapes , we do not know which shape we need until runtime.
- A factory class ShapeFactory is defined as a next step.
- FactoryPatternDemo, our demo class will use ShapeFactory to get a Shape object. It will pass information (CIRCLE / RECTANGLE / SQUARE) to ShapeFactory to get the type of object it needs.



```
#include <iostream.h>
```

```
#include <string>
```

```
class Shape {
```

```
public:
```

```
    Shape() {}
```

```
    virtual void draw() = 0;
```

```
};
```

```
class Circle : public Shape {
```

```
public:
```

```
    void draw() {
```

```
        cout << "Inside Circle::draw() method " ;
```

```
    }
```

```
};
```

```
class Square : public Shape {
```

```
public:
```

```
    void draw() {
```

```
        cout << "Inside Square::draw() method";
```

```
    }
```

```
};
```

```
class Rectangle : public Shape {
```

```
public:
```

```
    void draw() {
```

```
        cout << "Inside Rectangle::draw() method";
```

```
    }
```

```
};
```

```
class ShapeFactory {  
    public:  
    Shape getShape(string shapeType){  
  
        if(shapeType == "CIRCLE"){  
            return new Circle();  
        } else if(shapeType == "RECTANGLE"){  
            return new Rectangle();  
        } else if(shapeType == "SQUARE"){  
            return new Square();  
        }  
    }  
};
```

```
int main {  
    ShapeFactory shapeFactory;  
  
    //get an object of Circle and call its draw method.  
    Shape shape1 = shapeFactory.getShape("CIRCLE");  
    shape1.draw();  
  
    //get an object of Rectangle and call its draw method.  
    Shape shape2 = shapeFactory.getShape("RECTANGLE");  
    shape2.draw();  
  
    //get an object of Square and call its draw method.  
    Shape shape3 = shapeFactory.getShape("SQUARE");  
    shape3.draw();  
}  
}
```

Output

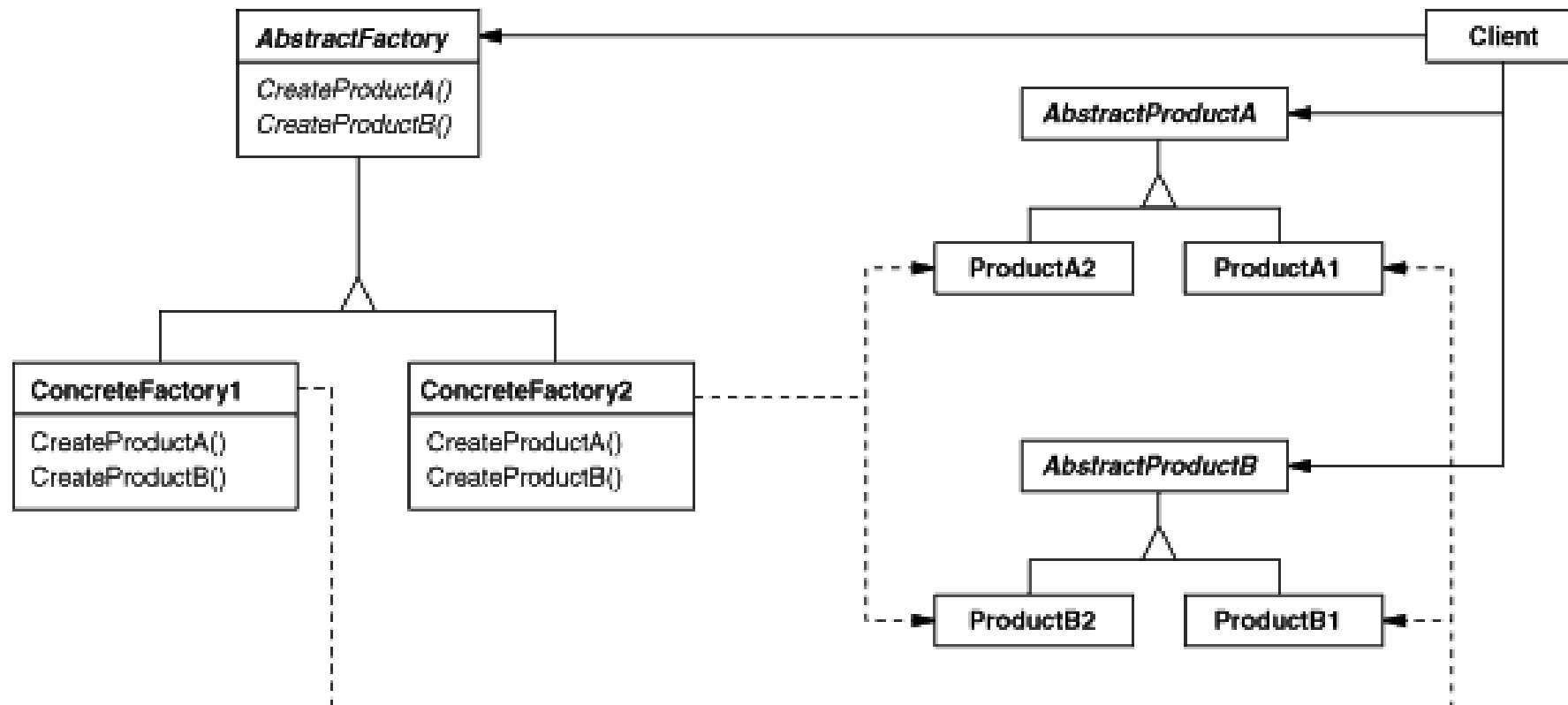
Inside Circle::draw() method.

Inside Rectangle::draw() method.

Inside Square::draw() method.

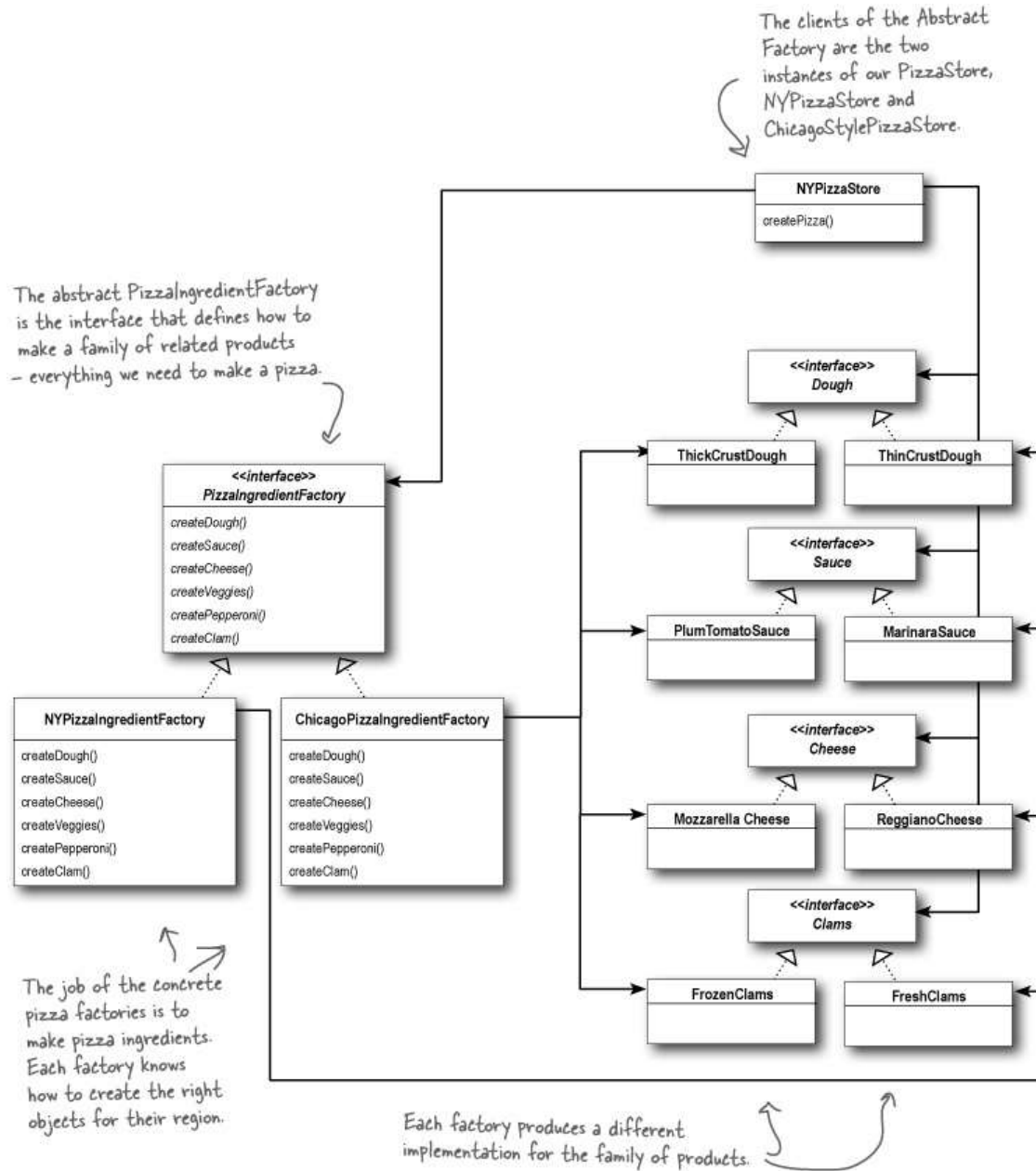
Abstract Factory Pattern

- Applicability
 - different families of components (products)
 - hide existence of multiple families from clients



Example

The abstract `PizzaIngredientFactory` is the interface that defines how to make a family of related products - everything we need to make a pizza.



Abstract Factory Pattern

- Abstract Factory patterns works around a super-factory which creates other factories.
- This factory is also called as Factory of factories.
- Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Factories** and **products** are the key elements to Abstract Factory pattern.
- In Abstract Factory pattern an interface is responsible for creating a factory of related objects, without explicitly specifying their classes.
- Each generated factory can give the objects as per the Factory pattern.

```
#include <iostream.h>
#define ROBUST
class Shape {
protected:
    int id;
    static int total;
public:
    Shape() {    id= total;  }
    virtual void draw() = 0;
};
int Shape::total= 0;

class Circle : public Shape {
public:
    void draw() {
        cout << "circle " << id<< ": draw" << endl;
    }
};
```

```
class Square : public Shape {
public:
    void draw() {
        cout << "square " << id<< ": draw" << endl;
    }
};

class Ellipse : public Shape {
public:
    void draw() {
        cout << "ellipse " << id << ": draw" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "rectangle " << id << ": draw" << endl;
    }
};
```

```
class Factory {  
    public:  
        virtual Shape* createCurvedInstance() = 0;  
        virtual Shape* createStraightInstance() = 0;  
};
```

```
class SimpleShapeFactory : public Factory {  
    public:  
        Shape* createCurvedInstance() {  
            return new Circle;  
        }  
        Shape* createStraightInstance() {  
            return new Square;  
        }  
};  
  
class RobustShapeFactory : public Factory {  
    public:  
        Shape* createCurvedInstance() {  
            return new Ellipse;  
        }  
        Shape* createStraightInstance() {  
            return new Rectangle;  
        }  
};
```

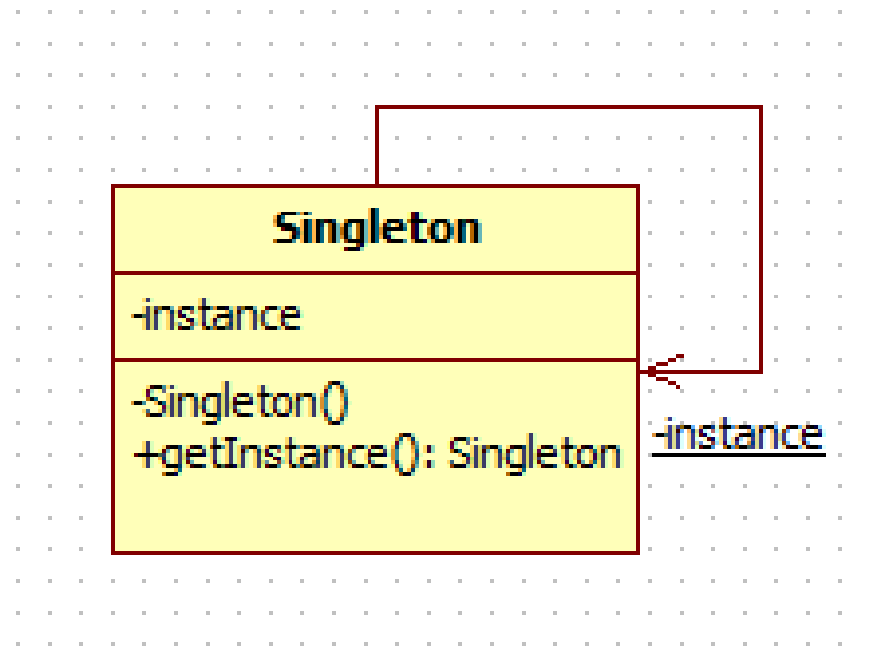
```
int main() {  
#ifdef SIMPLE  
    Factory* factory = new SimpleShapeFactory;  
#elif ROBUST  
    Factory* factory = new RobustShapeFactory;  
#endif  
  
    Shape* shapes[3];  
  
    shapes[0] = factory->createCurvedInstance(); // shapes[0] = new Ellipse;  
    shapes[1] = factory->createStraightInstance(); // shapes[1] = new Rectangle;  
    shapes[2] = factory->createCurvedInstance(); // shapes[2] = new Ellipse;  
  
    for (int i=0; i < 3; i++) {  
        shapes[i]->draw();  
    }  
}
```

Output
ellipse 0 : draw
rectangle 1 : draw
ellipse 2 : draw

Singleton objects

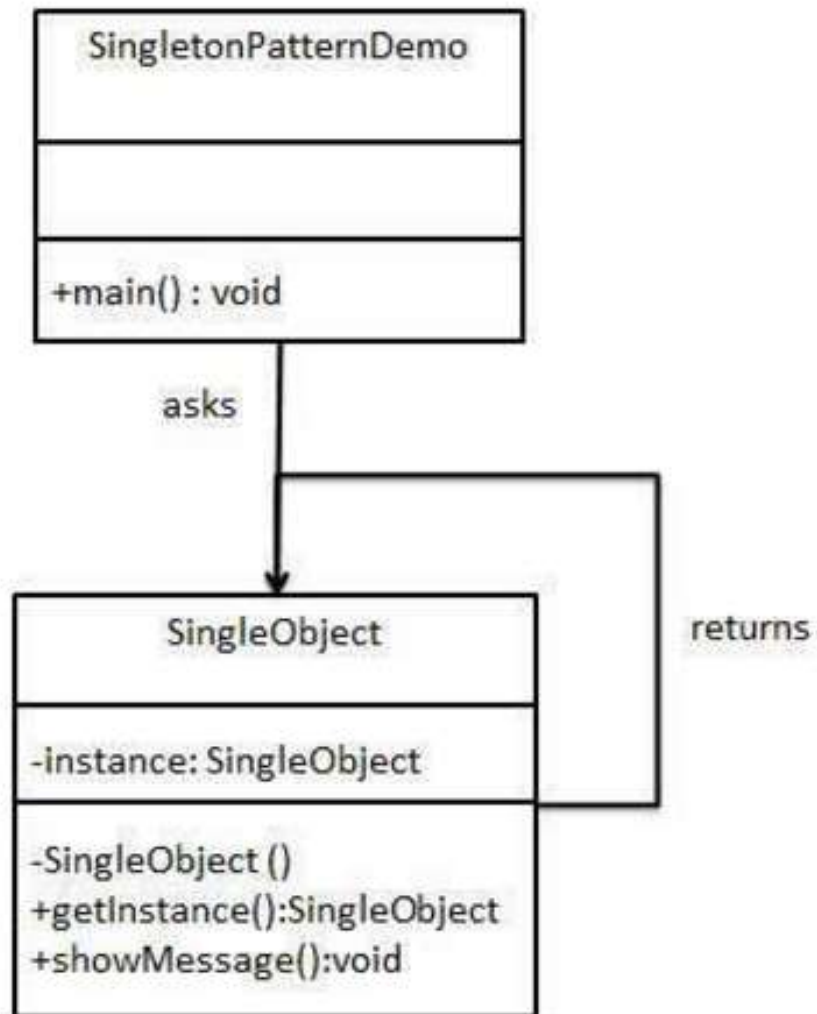
- Some classes have conceptually one instance
 - Many printers, but only one print spooler
 - One file system
 - One window manager
- Naïve: create many objects that represent the same conceptual instance
- Better: only create one object and reuse it
 - Encapsulate the code that manages the reuse

Singleton pattern



Singleton Pattern

- The Singleton pattern ensures a class has only one instance, and provides a global point of access to it.
- A Singleton is an elegant way of maintaining global state, but we should always question whether we need global state.
- This pattern involves a single class which is responsible to create its own object while making sure that only a single object gets created.
- This class provides a way to access its only object which can be accessed directly without the need to instantiate the object of the class.



```
class SingleObject {  
private :  
    SingleObject(){} //make the constructor private so that this class cannot be instantiated  
    static SingleObject instance; //create an object of SingleObject  
  
public:  
    //Get the only object available  
    static SingleObject getInstance(){ }  
  
    void showMessage(){  
        cout<<"Hello World!";  
    }  
}
```

```
Int main() {  
    SingleObject object ;  
    object.getInstance();  
  
    //show the message  
    object.showMessage();  
}  
}
```

Out put :

Hello World !

Thank You !